

Design Document: AI Powered Generic Data Transformation Service

1. Problem Statement

Organizations often receive input data from multiple sources in **heterogeneous formats** such as PDF, DOCX, and XLSX which needs to be converted to JSON for multiple downstream consumption, but each flow has its own logic and rules.

This requires building multiple systems and maintaining them

Key challenges include:

- **Lack of standardization** across input files
- **Manual and error-prone transformations**
- Difficulty in defining transformation logic for varying formats
- **Multiple systems required** for each pipeline.
- High effort required to integrate with downstream systems
- **Limited scalability** when onboarding new data formats or use cases

This results in **increased operational overhead, inconsistent data outputs**, delayed processing and for consuming systems.

2. Objective

Build a **generic, scalable AI-powered data transformation service** that:

- Accepts diverse input formats
 - Applies **user-defined transformation rules**
 - Produces **standardized JSON output**
 - Enables seamless integration with downstream systems
-

3. Proposed Solution

The solution introduces a **rule-driven, AI-powered transformation pipeline** that converts unstructured/semi-structured data into a standardized JSON format.

Core Concept

- Users define transformation logic via a **JSON-based schema (rules)**
 - This Schema will be provided along with input file.
 - The system uses **AI/LLM capabilities** to interpret and apply these rules
 - Output is validated and transformed into a **consistent structure**
-

4. High-Level Architecture Overview

Input Layer

- Accepts files: **PDF, DOCX, XLSX**
 - User provides:
 - Input file
 - JSON schema defining transformation rules
-

Processing Layer: AI-Powered Transformation Service

Step 1: Parse Input File

- Extract structured/unstructured data from:
 - Text content
 - Tables
 - Sections/pages

Step 2: Apply Rules (AI/LLM Engine)

- Interpret JSON schema
- Apply transformation logic
- Map extracted data to target structure
- Handle inconsistencies and ambiguities using AI

Step 3: Validate Output

- Validate output against:
 - JSON schema

- Business rules
 - Ensure completeness and correctness
-

Output Layer

- Generate **standardized JSON output**
 - Ensures:
 - Consistency across inputs
 - Compatibility with downstream systems
-

5. Key Components

Component	Description
Input Processor	Handles ingestion of documents (PDF/DOCX/XLSX)
Parser Engine	Extracts content (text, tables, metadata)
Rule Engine (JSON-based)	Defines transformation logic
AI/LLM Engine	Interprets rules and performs intelligent mapping
Validation Engine	Ensures output meets schema and quality standards
Output Formatter	Generates standardized JSON

6. Data Flow

1. User uploads input file and JSON schema
2. File is parsed to extract raw data
3. AI engine applies transformation rules
4. Output is validated against schema
5. Standardized JSON is generated
6. Output is consumed by downstream systems

7. Key Benefits

- Standardized data output across multiple formats
- Reduced manual effort and transformation errors
- Flexible rule-driven architecture
- Scalable for new formats and use cases
- Reusable across multiple applications/services